Course: Introduction to Artificial Intelligence
Instructor: Dr. Syed Ali Raza
Spring 2026
Institute of Business Administration, Karachi
Assignment #2

Duaa Naz - 30668

Evolutionary Optimization of Keyframe-Based Locomotion in a MuJoCo Environment

1. Overview

This report presents the implementation and results of an Evolutionary Algorithm (EA) designed to optimize keyframe-based locomotion for the HalfCheetah-v5 robot in the MuJoCo physics environment via Gymnasium. The goal is to evolve a sequence of torque keyframes that maximizes cumulative episode reward, encouraging the cheetah to move forward efficiently.

2. Problem Representation

A chromosome in this EA represents a complete locomotion policy. It consists of K keyframes, where each keyframe is a numpy array of 7 values: 6 torque values (one per joint, ranging from -1.0 to 1.0) and 1 duration value (a positive integer indicating how many timesteps the keyframe is applied). The keyframes are applied cyclically throughout the episode.

Chromosome structure:

- Chromosome = [KF1, KF2, ..., KFK]
- Each KFi = [torque1, torque2, torque3, torque4, torque5, torque6, duration]
- Torque values: sampled uniformly from [-1.0, 1.0]
- Duration values: sampled as integers from [10, 50]

3. Algorithm Design

3.1 Initialization

The population is initialized randomly. Each chromosome consists of K keyframes with random torque values drawn from a uniform distribution over [-1.0, 1.0] and random integer durations between 10 and 50 timesteps.

3.2 Fitness Evaluation

Each chromosome is evaluated by running it in the HalfCheetah-v5 environment for one complete episode. The keyframes are applied cyclically: when a keyframe's duration expires, the next keyframe in the sequence is applied. The cumulative reward over the entire episode is used as the fitness score. Rendering is disabled during training to maximize speed.

3.3 Parent Selection Methods

Three parent selection strategies were implemented and compared:

Binary Tournament Selection: Two individuals are randomly chosen from the population and the one with higher fitness is selected as a parent. This process is repeated to obtain the second parent.

Fitness Proportional Selection (Roulette Wheel): Each individual is assigned a selection probability proportional to its fitness. Since rewards are negative early in training, fitnesses are shifted by subtracting the minimum value before computing probabilities. This preserves population diversity.

Truncation Selection: The population is sorted by fitness and only the top 50% are eligible to be parents. Two parents are randomly drawn from this elite group. This applies strong selection pressure.

3.4 Crossover Methods

Two crossover operators were implemented:

Single-Point Crossover: A random cut point is chosen between keyframe positions. Child 1 receives keyframes before the cut from Parent 1 and keyframes after from Parent 2. Child 2 receives the complement.

Order Crossover (OX): A contiguous chunk of keyframes is copied from one parent into the child at the same positions. The remaining keyframe slots are filled with the other parent's keyframes in their original order.

3.5 Mutation Methods

Three mutation operators were implemented:

Gaussian Mutation: Each torque gene is independently mutated with probability 0.1 by adding Gaussian noise (mean=0, std=0.1), then clipped to [-1.0, 1.0]. The duration gene is also occasionally mutated with a small random integer offset, clamped to a minimum of 1.

Swap Mutation: Two randomly selected keyframes within the chromosome are swapped.

Insert Mutation: A randomly selected keyframe is removed from its position and reinserted immediately after another randomly selected keyframe.

3.6 Survival Selection

A $(\mu + \lambda)$ survival strategy is used: the current population and newly generated children are combined, and the top population-size individuals by fitness are retained for the next generation. This ensures elitism and guarantees fitness never decreases. Additionally, an optimization was applied: since the new population's fitness is already known from survival selection, it is reused in the next generation rather than re-evaluating, cutting total fitness evaluations roughly in half.

4. Experiments and Results

Multiple experiments were conducted varying the number of keyframes (K), population size, and generations. Four selection/crossover/mutation combinations were compared in each experiment:

- Combination 1: Binary Tournament + Single-Point Crossover + Gaussian Mutation
- Combination 2: Roulette Wheel + OX Crossover + Gaussian Mutation
- Combination 3: Truncation + Single-Point Crossover + Gaussian Mutation
- Combination 4: Binary Tournament + OX Crossover + Insert Mutation

4.1 Experiment 1: K=5, Population=10, Generations=20

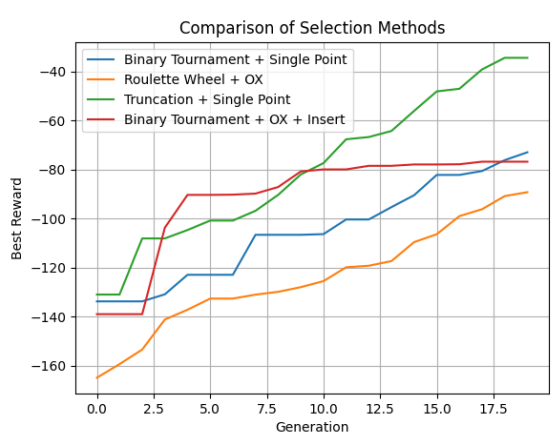

Figure 1: Convergence plot for K=5.

With 5 keyframes, Truncation + Single-Point Crossover achieved the best final reward of approximately -40, demonstrating that aggressive selection pressure is effective in smaller search spaces. Roulette Wheel performed worst, struggling with near-equal probabilities due to all-negative fitnesses early in training.

4.2 Experiment 2: K=6, Population=10, Generations=20

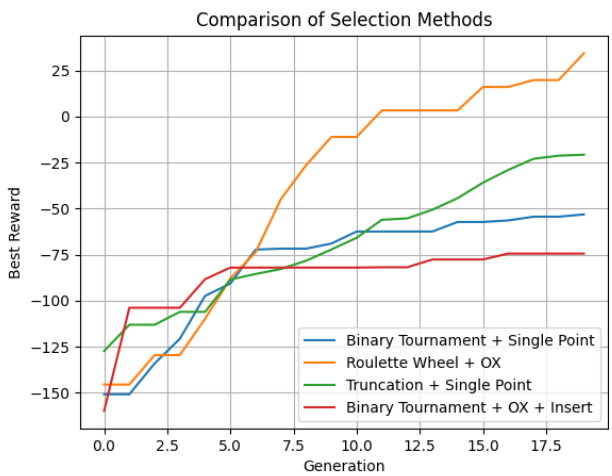

Figure 2: Convergence plot for K=6.

Increasing to 6 keyframes produced a dramatic shift: Roulette Wheel + OX Crossover achieved a positive reward of approximately +35, the best result observed across all experiments. The larger search space appeared to benefit from the diversity-preserving nature of roulette wheel selection. Notably, Truncation which won at K=5, fell behind, suggesting it may converge prematurely in larger spaces.

4.3 Experiment 3: K=7, Population=10, Generations=20

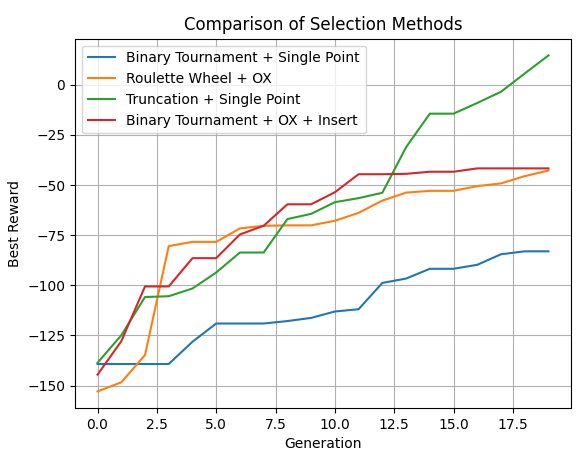

Figure 3: Convergence plot for K=7.

At K=7, Truncation reclaimed the top position with a final reward of approximately +10, while Roulette Wheel dropped to second. Binary Tournament + Single-Point consistently lagged across all K values, suggesting that single-point crossover without value-level mutation is too restrictive.

4.4 Experiment 4: K=8, Population=15, Generations=25

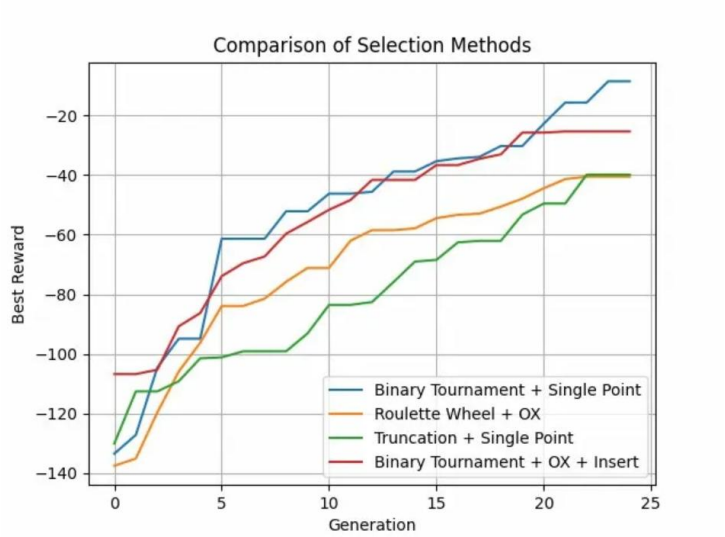

Figure 5: Convergence plot for K=8, Population=15, Generations=25.

The most revealing experiment: increasing population size and generations dramatically changed the rankings. Binary Tournament + Single-Point, previously the worst performer, achieved the best reward of approximately -15. This highlights that hyperparameter tuning can outweigh the choice of selection method.

5. Results Summary

Experiment	K (no of keyframes)	Population Size	Generations	Best Combination	Best Reward
Exp 1	5	10	20	Truncation + Single-Point	~-40
Exp 2	6	10	20	Roulette Wheel + OX	~+35
Exp 3	7	10	20	Truncation + Single-Point	~+10
Exp 4	8	15	25	Binary Tournament + Single-Point	~-15

6. Best Keyframes Found

The best keyframes found across all experiments (K=6, Roulette Wheel + OX, reward ~+35):

6

0.21, -0.26, -0.83, 0.42, -0.16, 0.16, 13.00

-0.17, 0.12, -0.01, 0.51, -0.52, 0.82, 39.00

0.34, 0.74, 1.00, -0.25, -0.47, 0.97, 6.00

-0.27, -0.90, 0.01, 0.69, -0.85, 0.84, 7.00

-0.17, 0.22, -0.08, -0.34, -0.52, -0.22, 12.00

-0.25, 0.94, 0.44, -0.01, -0.46, 0.95, 5.00

7. Analysis and Discussion

7.1 Effect of Number of Keyframes

The number of keyframes K has a significant impact on performance. Too few keyframes (K=5) limits the expressiveness of the locomotion policy, while too many may make the search space harder to navigate with small populations. K=6 achieved the best overall reward (+35), suggesting an optimal balance between policy expressiveness and search difficulty.

7.2 Selection Method Comparison

No single selection method consistently outperformed the others across all experiments. However, some patterns emerged:

- Truncation selection performed well at odd K values (5, 7) due to high selection pressure, but showed premature convergence at K=8.
- Roulette Wheel selection performed well at even K values (6, 8), benefiting from its diversity-preserving nature in larger search spaces.
- Binary Tournament was consistently average but became the best with larger population and more generations, suggesting it benefits most from computational budget.

7.3 Role of Hyperparameters

Experiment 4 demonstrated that population size and number of generations can have a greater impact than the choice of selection method. Increasing population from 10 to 15 and generations from 20 to 25 transformed the previously worst-performing combination into the best. This underscores the importance of hyperparameter tuning in evolutionary algorithms.

7.4 Convergence Behaviour

All combinations showed consistent improvement over generations, validating the EA implementation. Truncation selection exhibited the steepest early improvement but occasionally plateaued, while Roulette Wheel showed more gradual but sustained improvement. Binary Tournament showed the most stable, steady convergence curve.

8. Conclusion

This assignment successfully implemented an evolutionary algorithm to optimize keyframe-based locomotion for the HalfCheetah-v5 robot. The EA demonstrated clear improvement over random initialization across all combinations and experiments. The best result achieved was a positive reward of approximately +35 using Roulette Wheel selection with OX crossover and Gaussian mutation at K=6. Key findings include the importance of keyframe count, the context-dependent performance of selection methods, and the critical role of population size and generation count in determining convergence quality.